

Paper 6 - Causal Code

CQE-paper-06

CQECMPLX Formal Paper Series

Review-facing PDF built 2026-06-13

Construction Basis

Cast every dependency between objects, proofs, tools, and papers as a typed causal edge.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- top-level review source: Papers\Source\CQE-paper-06.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-06\paper_kernel_manifest.json
- verifier: production\formal-papers\CQE-paper-06\verify_causal_code.py
- receipt: production\formal-papers\CQE-paper-06\causal_code_receipt.json (Typed causal edge contract: pass)
- body: production\papers\CQE-paper-06\PAPER-BODY.md
- source: production\papers\CQE-paper-06\SOURCE.md
- formal: production\papers\CQE-paper-06\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-06\02-CQE-tool\TOOL.md
- workbook: production\papers\CQE-paper-06\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-06.md

Paper 6 - Causal Code

Abstract

Paper 6 proves the causal-edge contract for the CQECMPLX paper suite. Papers 1-5 define and transport local objects: carrier, correction, registration, repair, and path payload. Paper 6 defines how the dependencies among those objects must be recorded so that proof support, open obligations, and review loops cannot be confused.

The theorem is a graph discipline:

```
dependency -> typed causal edge
typed causal edge -> source, target, edge_type, receipt, status
closed proof support -> acyclic
open obligation -> visible, not proof
```

The result does not declare the full corpus complete. It proves the rule that prevents the corpus from hiding incompleteness.

Claims

Claim 6.1. Every production proof dependency must be represented as a typed causal edge with source, target, edge type, receipt, and status.

Claim 6.2. A causal edge is invalid if it lacks a receipt, uses an unknown edge type, or uses an unknown status.

Claim 6.3. Closed proof-support edges must be acyclic. Hidden proof cycles are rejected.

Claim 6.4. Open obligations remain open and may not be counted as proof closure.

Definitions

A **causal vertex** is a paper, proof, tool, receipt, obligation, package, or product artifact.

A **causal edge** is a record:

```
source
target
edge_type
receipt
status
```

Allowed edge types are:

```
uses
proves
refines
obligates
transports
repairs
constrains
verifies
```

Allowed statuses are:

```
open
closed
deferred
```

rejected

The proof-support edge types are:

uses
proves
refines
transports
repairs
constrains
verifies

An edge with status `open` is an obligation, not a proof closure.

Theorem 6.1 - Typed Causal Edge Contract

A paper-kernel dependency is admissible to the CQECMPLX production graph only if it is represented by a typed causal edge with source, target, edge type, receipt, and status. Closed proof-support edges must be acyclic. Open obligations must remain marked as open until a later receipt closes them.

Proof

Each required field is necessary.

Without `source`, the dependency has no origin. Without `target`, it has no claimed destination. Without `edge\_type`, the relation cannot be interpreted. Without `receipt`, the relation cannot be replayed. Without `status`, the graph cannot distinguish closure from obligation, deferral, or rejection.

Closed proof-support cycles are rejected because they allow a claim to support itself through the dependency graph. That is circular support, not proof. Cycles may exist as review, feedback, or obligation routing only when they are typed so they cannot masquerade as closed proof support.

Open obligations remain open by definition. Counting an open edge as closed changes the status field and violates the receipt. Therefore a valid causal graph can contain incompleteness, but it must expose it.

Concrete Graph From Papers 0-6

The first proof chain is:

CQE-paper-00	->	CQE-paper-01	uses	paper00-contract	closed
CQE-paper-01	->	CQE-paper-02	uses	lcr-carrier-receipt	closed
CQE-paper-02	->	CQE-paper-03	transports	correction-surface	closed
CQE-paper-03	->	CQE-paper-04	constrains	triality-surface	closed
CQE-paper-04	->	CQE-paper-05	repairs	boundary-repair	closed
CQE-paper-05	->	CQE-paper-06	transports	oloid-path-carrier	closed
CQE-paper-06	->	cqe_engine.formal	obligates	paper06-open-obligation	open

The closed proof-support portion is acyclic. The final open edge records an implementation obligation rather than pretending the interface binding is already complete.

Receipt

The primary executable receipt is:

```
production/formal-papers/CQE-paper-06/verify_causal_code.py
production/formal-papers/CQE-paper-06/causal_code_receipt.json
```

The receipt status is `pass`. It verifies:

```
all_edges_have_required_fields      = true
all_edges_have_allowed_type_and_status = true
closed_proof_support_graph_is_acyclic = true
open_obligations_remain_open       = true
missing_receipt_rejected           = true
unknown_type_rejected              = true
hidden_proof_cycle_rejected        = true
```

Falsifiers

The paper fails if any of the following are accepted:

```
an edge with no receipt
an edge with an unknown edge type
an edge with an unknown status
a hidden closed proof-support cycle
an open obligation counted as proof closure
```

Role in the Suite

Paper 3 registers states without losing structure. Paper 6 registers dependencies without losing proof responsibility.

The bridge is:

```
triality/Jordan registration supplies the registered object
causal code supplies the registered dependency
```

This is why causal code is not administrative decoration. It is the proof-preserving dependency algebra of the suite.

Open Obligations

- Expose `verify\_causal\_code` through the installable kernel/API interface.
- Populate the full 32-paper graph from all formal receipts.
- Replace placeholder receipt identifiers with repository-stable artifact hashes.
- Classify permitted review loops separately from rejected proof-support cycles.

Conclusion

Paper 6 proves the suite's dependency honesty layer. Every proof-support move must have a typed edge and a receipt. Closed proof support cannot hide cycles, and open obligations remain visible until a later receipt closes them.